After5 — Albert Architecture

One-liner: Lead fills form → Albert (AI) has a real WhatsApp conversation, qualifies them, and books a call with Louis. Every conversation is scored, logged, and visible on a live dashboard.

End-to-end flow

```
┌──────────────────┐
│  Website form    │
│  (name, phone,   │
│   message…)      │
└──────────────────┘
         │ POST /form-webhook
         ▼
┌──────────────────────────────────────────────┐
│  FastAPI app (main.py)                       │
│   ├─ New lead?       → send opening template │
│   └─ Returning lead? → welcome-back flow     │
│                        (keeps prior context) │
└──────────────────────────────────────────────┘
         │ sends via Baileys (WhatsApp)
         ▼
┌──────────────────┐        ┌──────────────────┐
│     Lead         │◄──────►│   Baileys svc    │
│   (WhatsApp)     │        │   (local :3001)  │
└──────────────────┘        └──────────────────┘
                                     │ inbound message
                                     ▼
┌──────────────────────────────────────────────┐
│  /webhook  →  Conversation Engine            │
│   ├─ Input buffer (5s — wait for typing)     │
│   ├─ Typing indicator                        │
│   ├─ State machine: Opening → Discovery →    │
│   │                 Qualification → Booking  │
│   ├─ LLM call (Claude Sonnet 4.5 via OpenRouter)│
│   ├─ Message chunker (2-3 bubbles, 1.5s delay) │
│   └─ Reply back via Baileys                  │
└──────────────────────────────────────────────┘
         │ after every turn (background)
         ▼
┌──────────────────────────────────────────────┐
│  Signal Scorer (GPT-4o-mini — cheap & fast)  │
│  prompts/bant_prompt.txt  (Louis's rubric)   │
│                                              │
│  Returns JSON:                               │
│    lead_gen, pain, intent, engagement        │
│       each: {score 0-10, evidence}           │
│    overall_score 0-10                        │
│    buying_signals: [...]                     │
│    recommended_action: continue_discovery |  │
```

```
    │                        push_for_booking │        │
    │                        escalate_to_human        │
    └─────────────────────────────────────────────────┘
            │
            │
            ▼

    ┌─────────────────┐       ┌──────────────────┐
    │   Supabase      │       │  Live dashboard  │
    │   (Postgres)    │◄──────►│  (Next.js 16)    │
    │                 │       │  localhost:3001  │
    │  leads          │       │                  │
    │  messages       │       │  Shows every     │
    │  conv_state     │       │  lead in real-   │
    │  bookings       │       │  time — chat,    │
    │  llm_sessions   │       │  signals, BANT,  │
    └─────────────────┘       │  buying signals  │
                              └──────────────────┘
```

Scoring — what every conversation produces

Every message exchange re-runs the scorer. Dashboard refreshes every 2.5s.

Category	What it measures	0–10
Lead gen	Do they actually generate inbound? Volume? Channels?	auto
Pain	Have they described a specific problem in their own words?	auto
Intent	Forward momentum — asking process, timeline, price, booking?	auto
Engagement	Responsiveness + detail — full answers or one-worders?	auto

Plus:

- **overall_score** (0–10) → drives lead temperature (Cold / Warm / Hot)
- **buying_signals[]** → short chips like "losing deals now", "founder is decision maker"
- **recommended_action** → what Albert should do next
  - `< 5` → keep doing discovery
  - `5–8` → push for the call
  - `≥ 9` → escalate, Louis should jump in personally

Decision rule (baked into the prompt): `lead_gen > 3 AND pain > 5 AND intent > 5` → ready to book. If any of those three is low → keep doing discovery regardless of overall score.

What makes Albert feel human (not a bot)

Mechanic	What it does
Input buffer	Waits 5s for lead to finish typing before replying
Typing indicator	Fires before every reply
Per-char typing delay	Longer messages "take longer to type" — feels real

Message chunking	Splits long replies into 2–3 WhatsApp bubbles, 1.5s apart
Returning-lead memory	Re-submits don't restart discovery — injects prior context
Template variety	3 randomized welcome-back lines + opening variants
24h session auto-close	Stale sessions re-open with a natural "hey again" vs cold open
AI disclosure	If asked "are you a bot?" → honest, casual answer
Personality matching	Conversation library covers driver / amiable / analytical / expressive
Graceful exits	Difficult / abusive / not-interested leads handled cleanly

The system prompt (`Mater Doc/After5_Master_System_Prompt.md`) explicitly forbids corporate language, padding, and performative summarization — the rhythm comes from restraint, not vocabulary.

Albert's voice — real examples

These aren't mocked — they're pulled straight from the conversation library and templates the system uses in production.

Cold opening (new lead from form)

> *Hey Aisha, Albert here from After5. Just saw your demo request come through for BrightMedia, so thought I'd get in touch.* ***What was it that made you want to check it out?***

Short. Casual. No corporate "I hope this message finds you well." One open question. Gets them talking.

Welcome-back (returning lead — they filled the form a second time)

> *Hey Aisha, Albert here again from After5. Saw your name come through again for BrightMedia,* ***good to have you back. What's changed since last time?***

Randomized across 3 variants so it never feels templated. Doesn't restart discovery — injects prior-conversation context into the LLM so it remembers what they talked about.

"Are you a bot?"

> ***lead:*** *wait are you a real person*
>
> ***Albert:*** *haha no, I'm AI actually. Albert, After5's sales agent. Same job as any rep though, just figuring out if we can actually help. What's your situation?*
>
> ***lead:*** *oh ok thats actually kinda cool*
>
> ***Albert:*** *Yeah it's basically what we build for other businesses. So you're getting a live demo right now.* ***What kind of leads are you working with?***

Honest. Disarms the objection in one line. Then uses the reveal as a sales point.

Price objection (pivot to booking)

> ***lead:*** *I want to know the price first. What's the damage?*

> ***Albert:*** *hard to give a number here as it's scope-based, depends what needs building. but usually it comes in under a month of your ad spend*
>
> *[next bubble] we split it 50/50 too, so it's not all upfront. Louis can give you an exact figure once he sees your setup*
>
> ***lead:*** *That doesn't tell me much though.*
>
> ***Albert:*** *look, roughly it's a few grand for the build, but honestly it depends if we're building for one channel or five*
>
> *[next bubble] it's easier to just grab 20 mins with Louis. he'll give you a proper figure straight away and you can decide if it makes sense.* ***want the link?***
>
> ***lead:*** *Alright, send it.*

Notice the two-bubble chunking (`|||` splits) — feels like someone typing a thought, pausing, then adding a second line. The second attempt doesn't lecture — it concedes, reframes, closes.

No-stress follow-up (day 2 if they ghost)

> *Hey Aisha, just following up from yesterday. No stress if the timing's off, just didn't want you to miss the message. Maybe if we would need this or not, lmk*

Low pressure. No "hope to hear from you soon!" No guilt. Humans actually reply to this — bots get ignored.

The anti-patterns Albert never does

Explicitly forbidden in the system prompt (`Mater Doc/After5_Master_System_Prompt.md`):

- ❌ "Based on what you've shared, I believe a conversation with our founder Louis would be highly beneficial."
- ❌ "I understand your concerns regarding pricing. Let me elaborate on our value proposition."
- ❌ Summarizing what the lead just said back to them.
- ❌ Multiple clauses where one would do.
- ❌ Three-paragraph replies when one line works.

The rule in the prompt: *"after writing a reply, look at it again and remove anything that doesn't change the meaning. Most messages can lose 30–50% of their words and become clearer."*

Live dashboard (what Louis sees)

- **Left rail:** every conversation, sorted by activity, with phase + temperature + last message preview.
- **Center:** full WhatsApp-style chat transcript, updates live.
- **Right panel:**
  - Lead details (name, phone, email, company, industry, source)
  - Form message (what they originally wrote)
  - **Status** — phase, temperature, outcome, overall score, message count
  - **Signals** — Lead gen / Pain / Intent / Engagement with per-category score + evidence
  - **Action** — Continue Discovery / Push For Booking / Escalate To Human
  - **Buying signals** — chips of the specific phrases that flipped the meter
  - Booking info (scheduled time + status) if a call was booked
- **Reset button** — wipes state and re-fires opening template for clean test runs.

Tech stack

Layer	Choice	Why
Backend	FastAPI (Python 3.11, async)	Low-latency, async-first
LLM (primary)	Claude Sonnet 4.5 (OpenRouter)	Best for natural conversation
LLM (fallback)	GPT-4o (OpenRouter)	Reliability if Claude is down
LLM (scoring)	GPT-4o-mini (OpenRouter)	Cheap + fast, runs on every message
Observability	Helicone (optional proxy)	Per-conversation LLM cost + latency
WhatsApp transport	Baileys (Node.js, local service)	Stable unofficial WA interface
Session state	Redis	Input buffering, conversation memory
Persistent data	Supabase (Postgres)	Leads, messages, state, bookings
Dashboard	Next.js 16 + Tailwind v4	Fast, real-time, easy to iterate
Deployment	Railway (current) / Vercel-ready	Push to deploy

Data model (Supabase)

- **leads** — person + contact info + overall score + temperature + outcome
- **messages** — every inbound/outbound, with direction + timestamp
- **conversation_state** — per-lead phase, 4 signal scores + evidence, buying signals, recommended action
- **bookings** — Calendly event ID + scheduled time + status
- **llm_sessions** — per-LLM-call cost, token count, latency (for observability)
- **training_data** — exported conversations with human-reviewed scores (for future fine-tuning)

Per-message lifecycle (what happens in 8 seconds)

1. Lead sends a WhatsApp message → Baileys → `POST /webhook`.
2. Message goes into Redis input buffer (5s window in case they're still typing).
3. Buffer flushes → typing indicator sent to the lead.
4. Conversation engine builds context: system prompt + history + returning-lead notes.
5. Primary LLM (Claude Sonnet 4.5) generates reply (~1–2s).
6. Reply chunked → sent in 2–3 WhatsApp bubbles with 1.5s delays → feels typed.
7. Background: scoring LLM (GPT-4o-mini) re-scores the whole conversation.
8. Supabase gets the new message + updated signals → dashboard repaints within 2.5s.

Why this matters (the business case for Louis)

Without Albert	With Albert
Leads wait 20min–4h for first reply	Reply in <10s, 24/7
Manual qualification by VAs, inconsistent	Consistent signal scoring on every conversation

Hot leads miss the booking window	Hot leads auto-escalate, Louis jumps in personally
No visibility until VA updates a spreadsheet	Live dashboard with temperature + signals
Sales knowledge trapped in Louis's head	Captured in the system prompt + conversation library

The promise: every single website lead gets a real, natural, personalized WhatsApp conversation within seconds of hitting submit — and Louis walks into every call knowing exactly what the lead said, what they want, and how hot they are.